

CSCI 8205 Milestone 1 Report

Multi-GPU Pipeline Architecture for Real-Time Digital Inline Holographic Reconstruction

Course: CSCI 8205: Design and Implementation of Multiprocessor Systems

Students: Nicholas Bravo-Frank (bravo095@umn.edu), Anlei Chen (chen8264@umn.edu)

Date: March 5, 2026

Reporting Period: February 11 - March 5, 2026 (Weeks 1-4, with extensions into Weeks 5-6)

***Note:** This document was formatted and edited using Claude Sonnet 4 (claude-sonnet-4-20250514). Code examples, figure organization, and technical analysis were refactored and validated using Claude assistance.*

Summary

This milestone report documents progress on developing a parallel (multi-GPU) pipeline architecture for real-time digital inline holographic reconstruction. We have exceeded our original Week 1-4 timeline targets and completed work planned through Week 6, achieving a **7.8× end-to-end performance improvement** over our Python baseline with the C++ TensorRT implementation (6.3 ms vs 49.3 ms per frame).

Progress

- **C++/CUDA reconstruction kernel** completed and validated against Python reference

- **Full 6-stage pipeline implementation** in C++ with native TensorRT integration
- **Comprehensive benchmarking framework** using 10,000 frames of experimental data (1440×1080 grayscale images)
- **Multi-architecture comparison:** Python CPU/GPU vs C++ CPU/GPU implementations
- **Roofline analysis** with bandwidth ceilings and in-core performance limits
- **OpenMP CPU baseline** with thread scaling analysis (1-64 cores)

Key Performance Results

- **End-to-end latency:** 6.3 ms (158 FPS) sustained over 10,000 frames
- **Real-time target:** 67% progress toward 400 Hz camera target (2.5 ms/frame)
- **Stage-level improvements:** Up to 56.8× speedup in holographic reconstruction
- **Implementation robustness:** P95/median latency ratio of only 1.08× (excellent predictability)

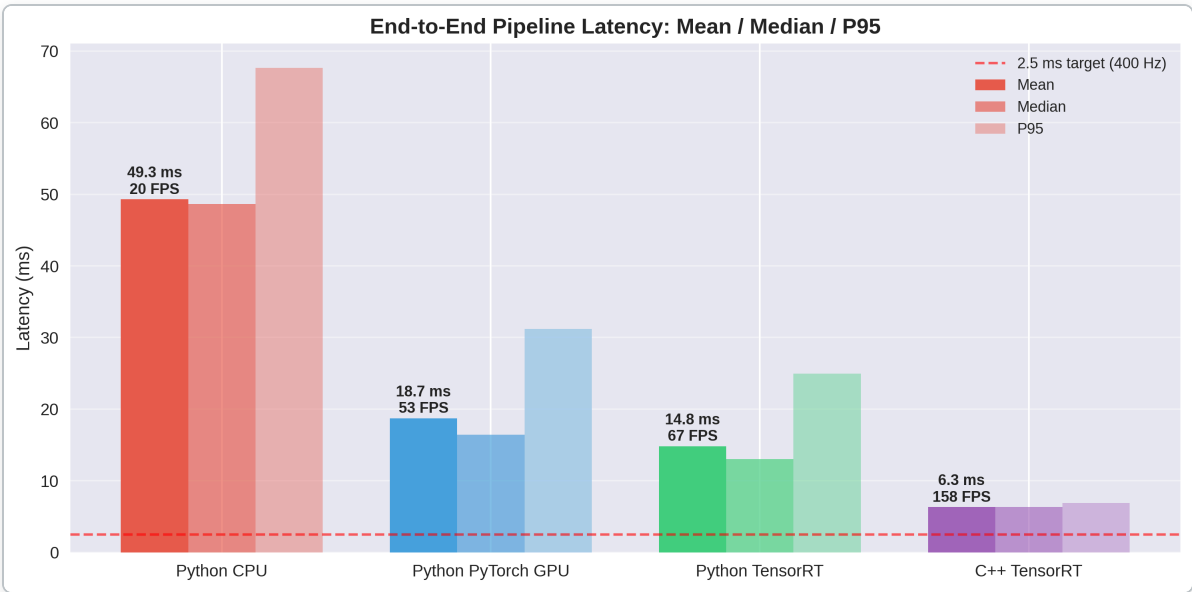


Figure 1: End-to-end pipeline performance comparison across implementations

1. Project Background and Objectives

1.1 Problem Statement

Digital inline holography (DIH) reconstructs three-dimensional particle fields from two-dimensional interference patterns through a six-stage computational pipeline:

1. **Image Load & Transfer** (~1 ms) - Disk read, resize, host-to-device transfer; synthetic camera read to RAM and move to GPU from buffer
2. **EMA Background Subtraction** (~3 ms) - Exponential moving average enhancement
3. **Multi-plane Reconstruction** (~1 ms/plane) - Angular spectrum propagation via 2D FFT (outperforms fresnel propagation on GPU)
4. **ML-based Particle Detection** (~3-4 ms) - CNN heatmap generation (batch=32, model developed previously and not part of this work)
5. **ML Classification** (~1 ms) - Per-particle crop classification (batch=32, YoloV21 classifier model, also not part of this work)
6. **Post-processing** (~1-10 ms) - Temporal consistency filtering

The challenge is designing a parallel workflow and multi-GPU system that can (1) sustain the core pipeline at maximum throughput while (2) concurrently processing computationally expensive autofocus workloads (10-100× slower than individual pipeline stages) without stalling frame ingestion.

1.2 Original Schedule and Targets

Per our proposal, **Week 1-4 objectives** were:

- Port angular spectrum propagation to C++/CUDA with cuFFT
- Implement OpenMP CPU baseline with FFTW (1-32 cores)
- Validate implementations against Python reference
- Initial roofline analysis characterizing compute vs memory-bound regimes
- **Deliverable:** Validated reconstruction kernels with initial roofline plots

1.3 Hardware Platform

- **GPU:** NVIDIA RTX 5090 (Blackwell, 170 SMs, 105 TFLOPS FP32, 1790 GB/s GDDR7, 32GB VRAM)
- **CPU:** AMD Threadripper PRO 7975WX (32 cores, Zen 4, 150 GB/s DDR5)
- **Development environment:** Ubuntu 24.04, CUDA 13.1, PyTorch 2.10.0+cu130

2. Implementation Progress

2.1 Completed Software Components

2.1.1 C++/CUDA Holographic Reconstruction

- **Location:** `cpp_cuda/src/holographic_reconstruction.cu`
- **Features:**
 - cuFFT-based 2D FFT with batched inverse transforms
 - Hz_stack caching optimization (compute forward FFT once, reuse across depth planes)
 - Fused normalization and cropping kernels
 - GPU memory pooling for zero-allocation operation
- **Validation:** Verified against Python reference across 20 configurations (4 image sizes $\times$ 5 depth counts)
- **Performance:** 2.8 $\times$ faster than PyTorch GPU at pipeline operating point (448 $\times$ 448, 3 planes)

2.1.2 OpenMP CPU Baseline

- **Location:** `cpp_cuda/src/cpu_baseline.cpp`
- **Implementation:** FFTW3 with OpenMP parallelization
- **Thread scaling:** Tested 1, 2, 4, 8, 16, 32, 64 threads
- **Key finding:** NUMA topology limits scaling - performance collapses at 64 threads due to cross-CCD cache coherence thrashing (best guess, need to validate)

2.1.3 Complete 6-Stage C++ Pipeline

- **Location:** `cpp_cuda/src/pipeline.cu`, `cpp_cuda/src/benchmark_pipeline.cu`
- **Integration:** Native TensorRT C++ API (not torch2trt wrapper)
- **ML Models:**
 - Heatmap detection: MobileNetV3-Small + FPN

- Classification: YOLO11n-cls
- **Memory management:** Pre-allocated GPU buffers (~80MB total), zero runtime allocation

2.1.4 Benchmarking Framework

- **Coverage:** 4 implementations × 240+ configurations per implementation
- **Statistical robustness:** 10,000-frame sustained runs for C++ TensorRT
- **Metrics:** Per-stage latency (mean/median/P95/P99), throughput, memory utilization
- **Cross-validation:** Automated numerical validation across all implementations

2.2 Performance Analysis Infrastructure

2.2.1 Extended Roofline Model

- **Multi-level bandwidth ceilings:** L1/Shared (~12 TB/s), L2 (~6 TB/s), GDDR7 (1790 GB/s)
- **In-core computation ceilings:** Peak FP32 (105 TF), No FMA (52.5 TF), Typical cuFFT (31.5 TF)
- **Locality analysis:** L1→L2 and L2→GDDR7 transition points for working set analysis

2.2.2 Cross-Implementation Validation

- **Numerical accuracy:** CUDA ↔ OpenMP < 1e-5, C++ ↔ Python < 1e-2
- **Performance correlation:** Consistent scaling trends across 4+ orders of magnitude problem size
- **Automated testing:** 20+ configuration validation suite with configurable tolerances

3. Key Results and Findings

3.1 End-to-End Performance Achievements

Implementation	Mean (ms)	Median (ms)	P95 (ms)	FPS	vs CPU Baseline
----------------	-----------	-------------	----------	-----	-----------------

Python CPU	49.3	48.6	67.7	20	1.0×
Python PyTorch GPU	18.7	16.4	31.2	53	2.6×
Python TensorRT	14.8	13.1	25.0	67	3.3×
C++ TensorRT	6.3	6.4	6.9	158	7.8×

Notes: The C++ TensorRT pipeline demonstrates high latency predictability (P95/median = 1.08×) compared to Python TensorRT (P95/median = 1.91×), essential for real-time applications and workloads.

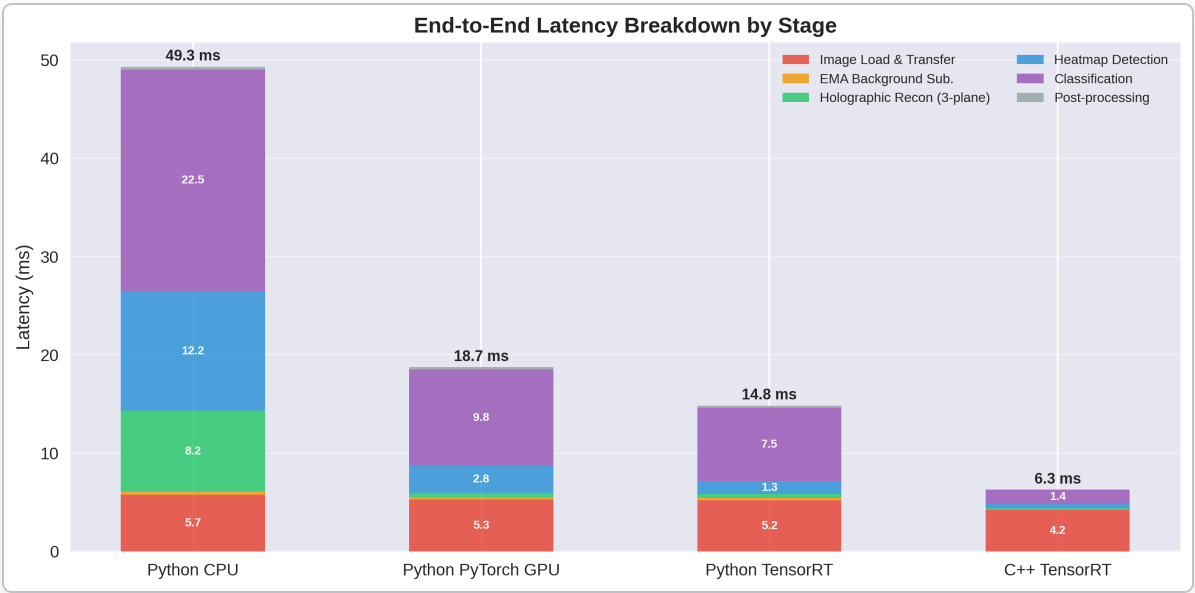


Figure 2: Pipeline stage performance breakdown showing bottleneck transformation

3.2 Stage-by-Stage Performance Analysis

3.2.1 C++ vs Python TensorRT Direct Comparison

Stage	Python TRT	C++ TRT	Speedup	Primary Cause
Image Load	5.19 ms	4.22 ms	1.2×	Pinned memory vs cv2 default
EMA	0.256 ms	0.007 ms	38.2×	Fused CUDA kernel vs PyTorch ops
Reconstruction	0.408 ms	0.145 ms	2.8×	Hz_stack caching, fused kernels

Detection	1.287 ms	0.457 ms	2.8×	Native TRT API vs torch2trt wrapper
Classification	7.478 ms	1.447 ms	5.2×	Direct TRT vs ultralytics overhead
Post-processing	0.218 ms	0.016 ms	13.9×	C++ std::vector vs Python lists

Notes: Largest speedups result from eliminating Python/framework overhead rather than algorithmic improvements.

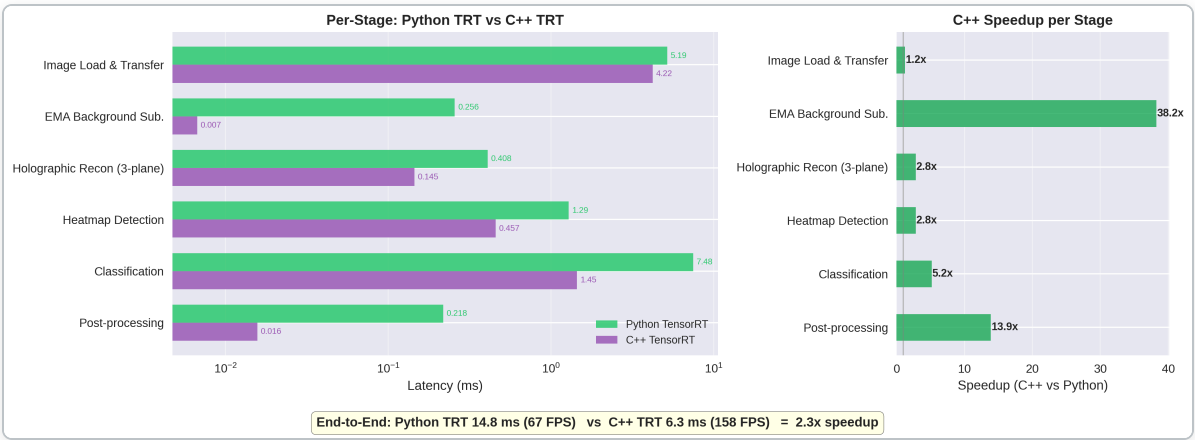


Figure 3: Direct comparison between C++ and Python TensorRT implementations

3.2.2 Compute Architecture Comparison

Per-stage speedup over Python CPU baseline:

- **Holographic Reconstruction:** 56.8× (FFT-dominated, *easily* parallelized)
- **EMA Background:** 49.2× (Memory bandwidth bound, ideal for GPU)
- **Heatmap Detection:** 26.6× (CNN inference highly parallel)
- **Classification:** 15.6× (Limited by variable batch size 0-50 crops/frame)
- **Image Load:** 1.4× (Disk I/O bound, GPU cannot accelerate but there may be a clever way to load from real/simulated camera stream)

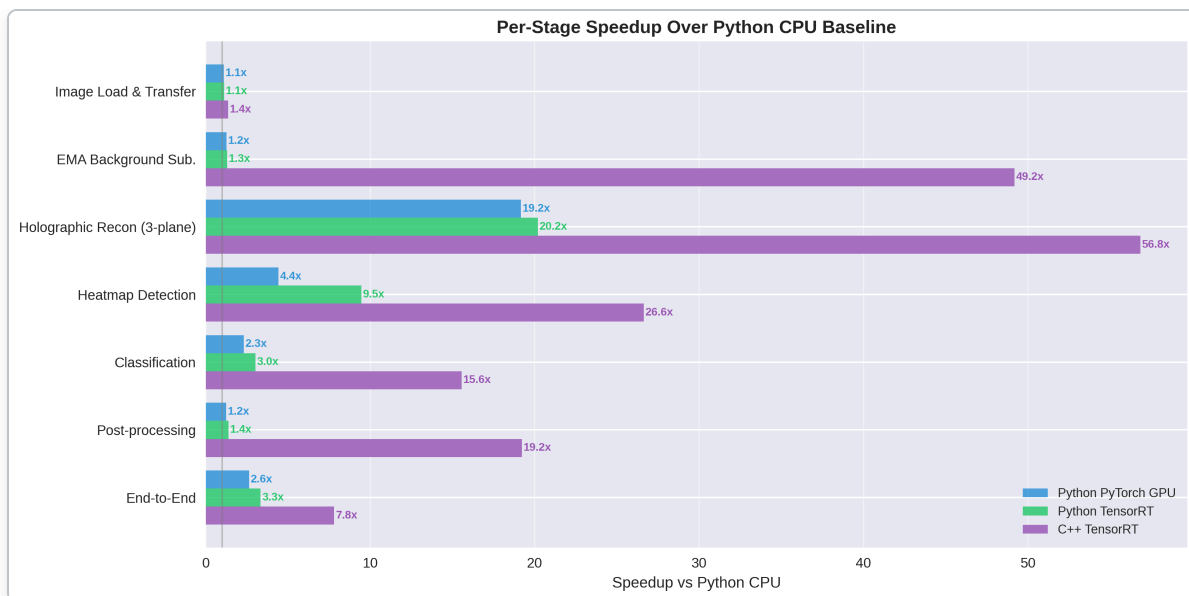


Figure 4: Per-stage speedup analysis revealing compute characteristics

3.3 Roofline Analysis Results

3.3.1 GPU Pipeline Stages on Extended Roofline

Stage	AI (FLOP/byte)	Achieved	% of Roofline	Performance Regime
Load & H2D	0.2	0.05 GF/s	0%	I/O bound (disk bottleneck)
EMA	0.6	258 GF/s	26%	Memory bandwidth bound
Reconstruction	10.0	1,085 GF/s	6%	Below cuFFT ceiling
Heatmap (TRT)	104	1,313 GF/s	1.3%	Compute bound, under-utilized
Classification (TRT)	202	1,866 GF/s	1.8%	Compute bound, batch overhead
Post-processing	0.2	0.4 GF/s	0%	CPU scalar operation

Notes: EMA is the only stage operating efficiently relative to its roofline position (26%). TensorRT stages have high arithmetic intensity (>100 FLOP/byte) but achieve only 1-2% of theoretical peak, indicating significant GPU under-utilization that motivates concurrent execution via CUDA streams.

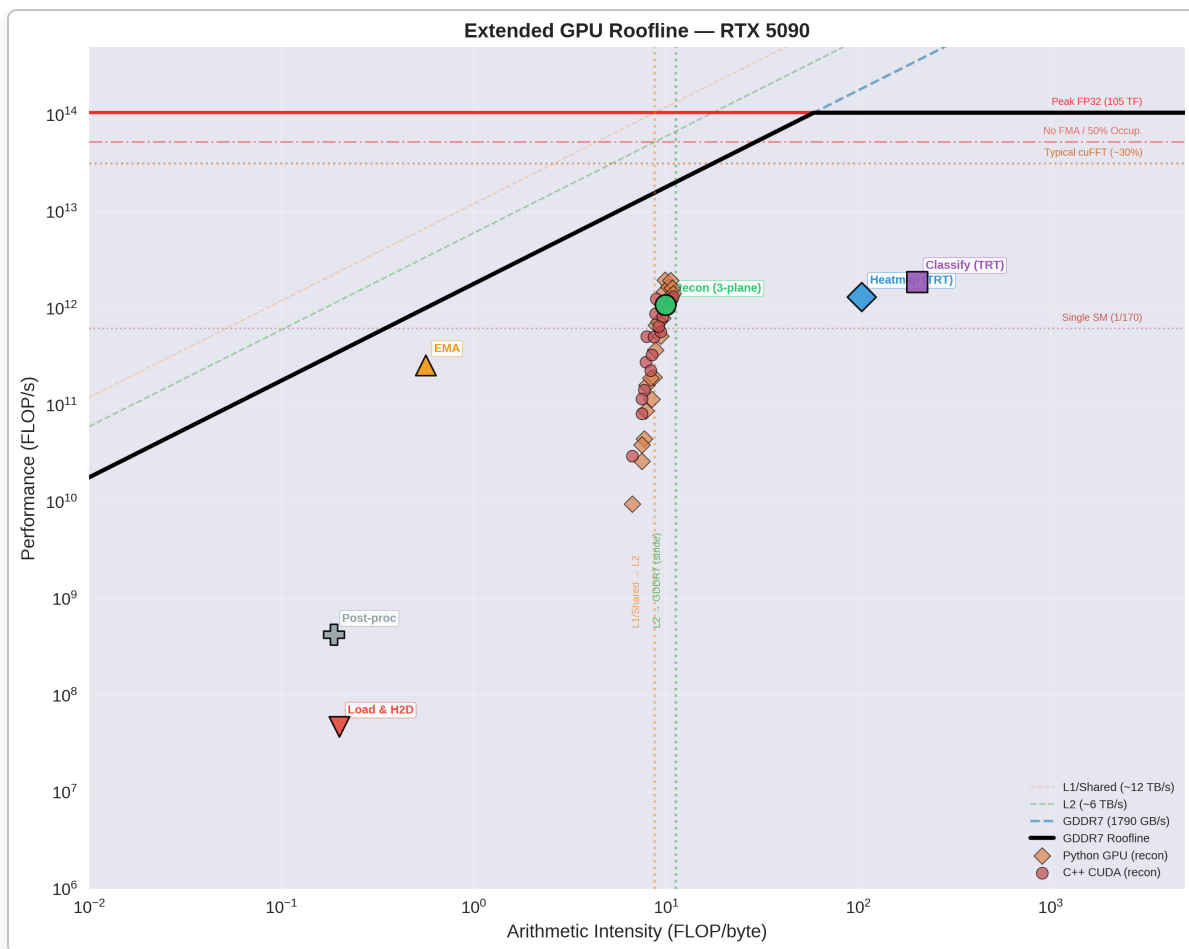


Figure 5: Extended GPU roofline analysis with multi-level bandwidth ceilings

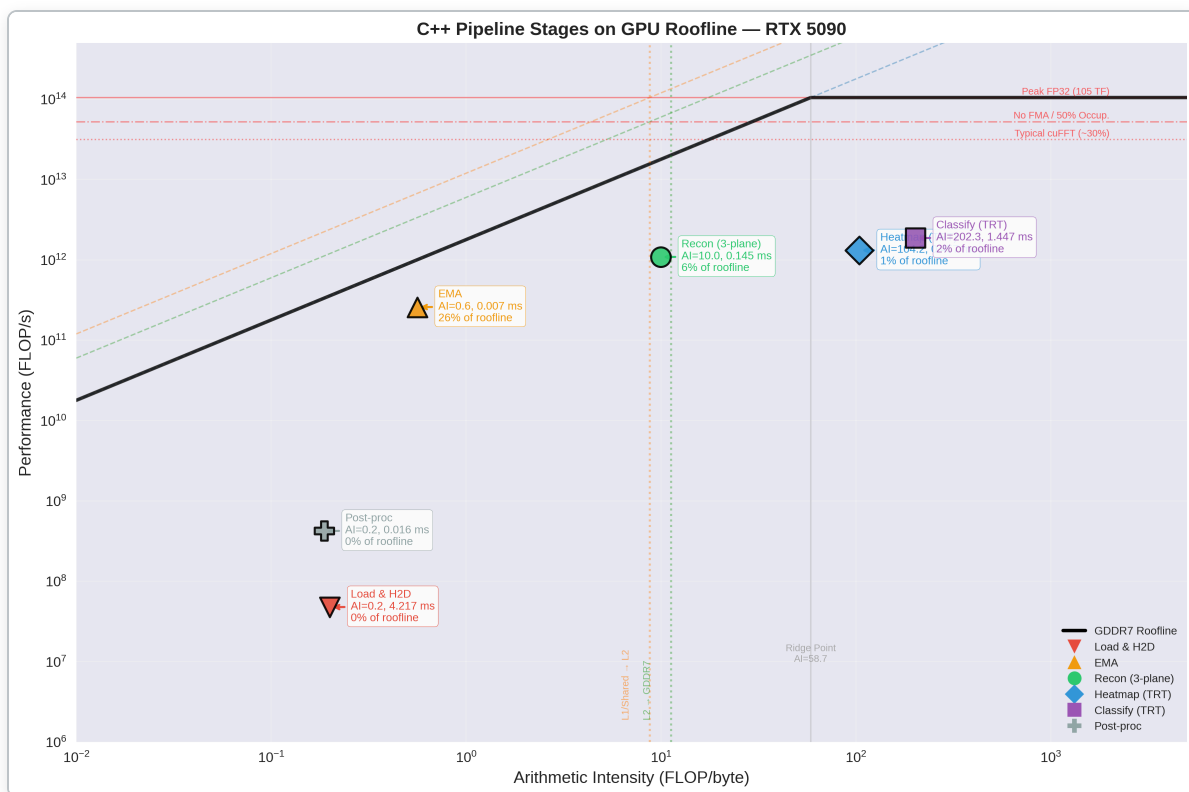


Figure 6: Individual pipeline stages plotted on GPU roofline model

3.3.2 CPU OpenMP Thread Scaling

Scaling efficiency on Threadripper PRO 7975WX:

- **Small problems (128×128):** Poor scaling due to overhead, best performance at 32 threads with only 1.3-2.6× speedup
- **Large problems (1024×1024):** Better scaling up to 7.4× with 32 threads (1 plane), 6.3× (20 planes)
- **64 threads:** Performance collapse due to NUMA effects across 4 CCDs

Root cause: Memory bandwidth saturation at ~150 GB/s DDR5 combined with 2D FFT's poor cache line utilization during strided column-pass operations.

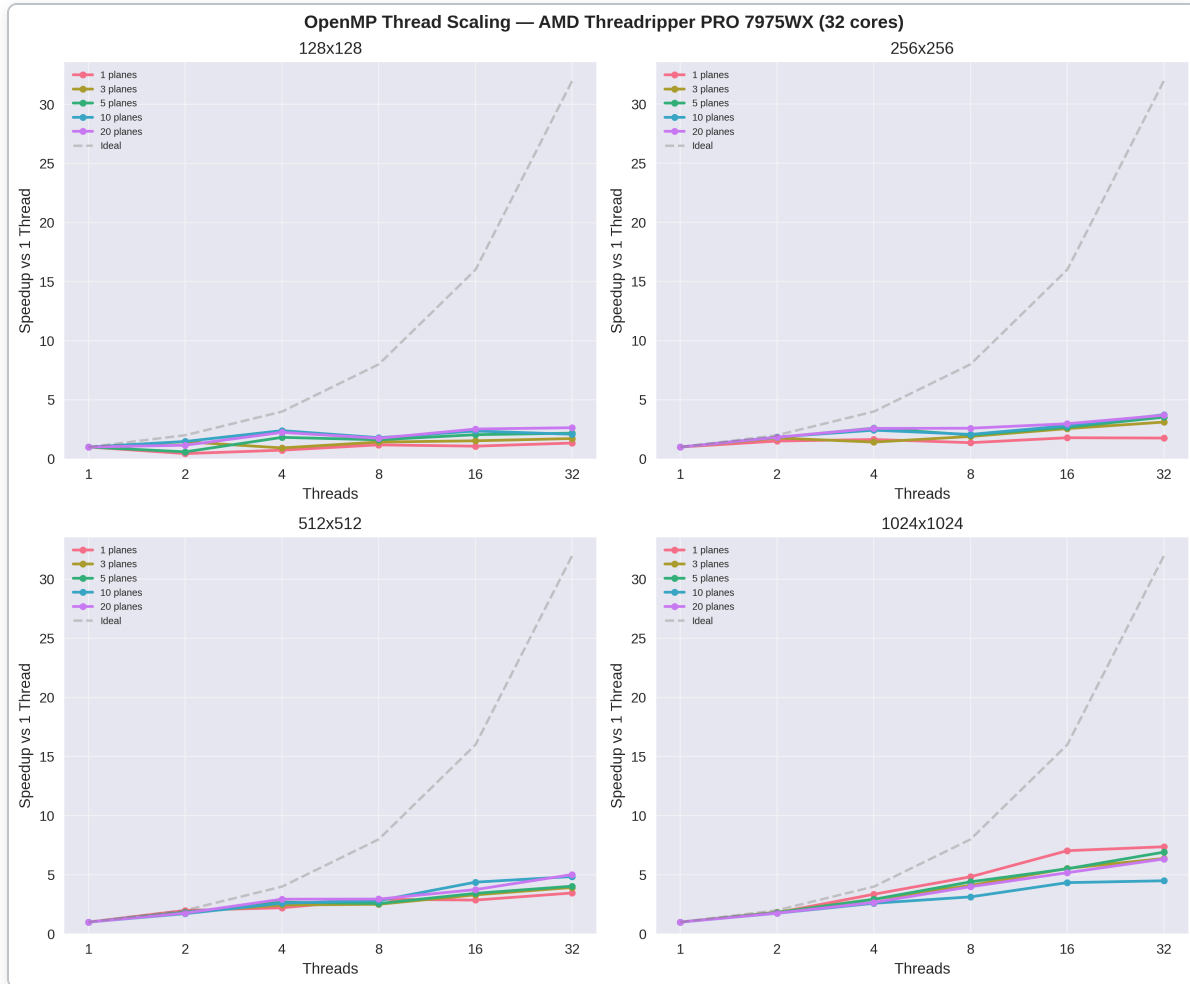


Figure 7: OpenMP thread scaling efficiency on Threadripper NUMA topology

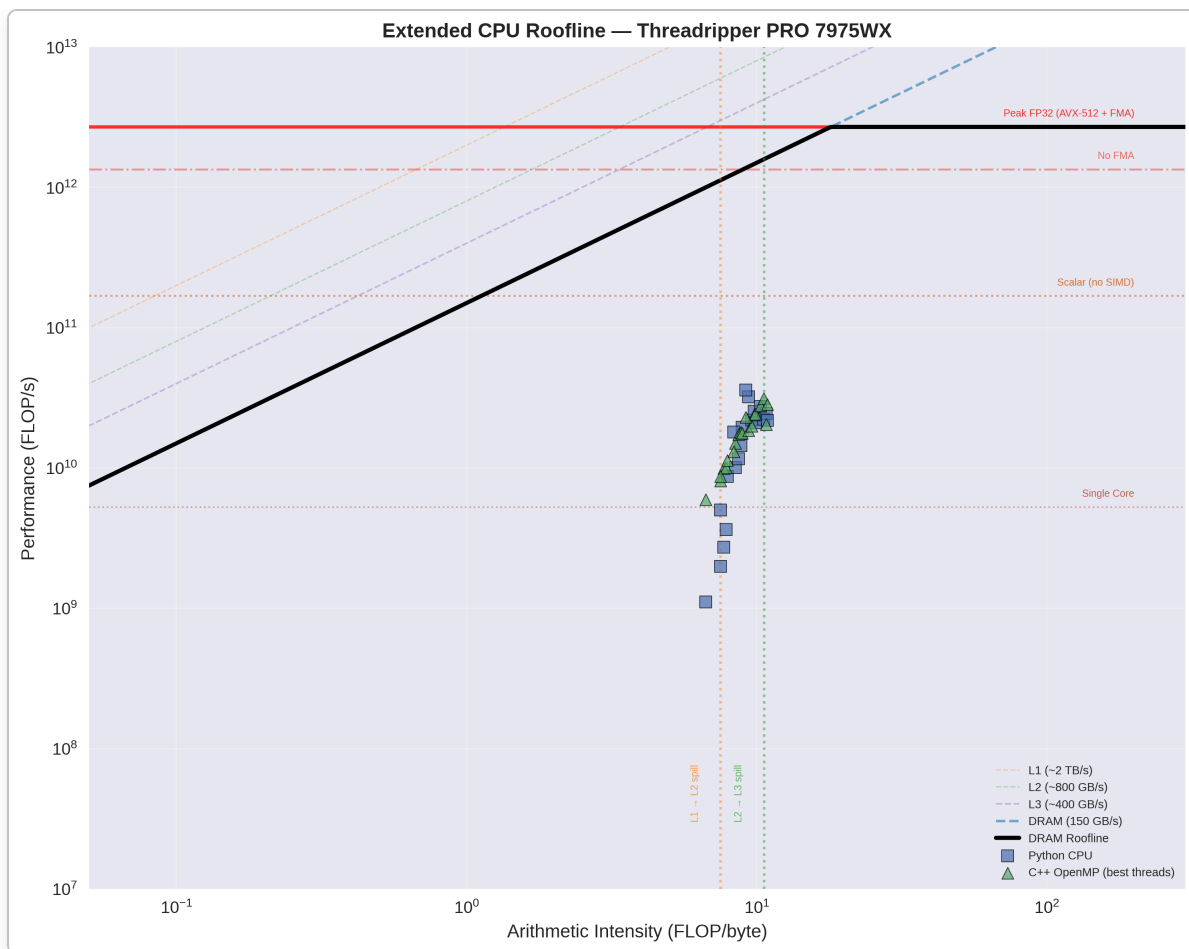


Figure 8: CPU roofline analysis revealing memory bandwidth saturation

3.4 Architectural Insights

3.4.1 Reconstruction Size Scaling Crossover

C++ CUDA is only faster than Python GPU at small image sizes.

Configuration	Python GPU (ms)	C++ CUDA (ms)	C++ Advantage
128×128 × 1p	0.28	0.09	3.1×
512×512 × 5p	0.33	0.32	1.0×
1024×1024 × 20p	3.25	3.49	0.9× (slower)

Possible Explanation: PyTorch's cuFFT path is highly optimized for large FFTs with better stream management and L2 cache utilization. C++ implementation's explicit synchronization and batched operations show worse cache

behavior at large sizes.

Pipeline relevance: At the operating point ($448 \times 448 \times 3$ planes), C++ maintains a $2.8\times$ advantage, but autofocus workloads (100-1000 planes) may require L2-aware tiling strategies.

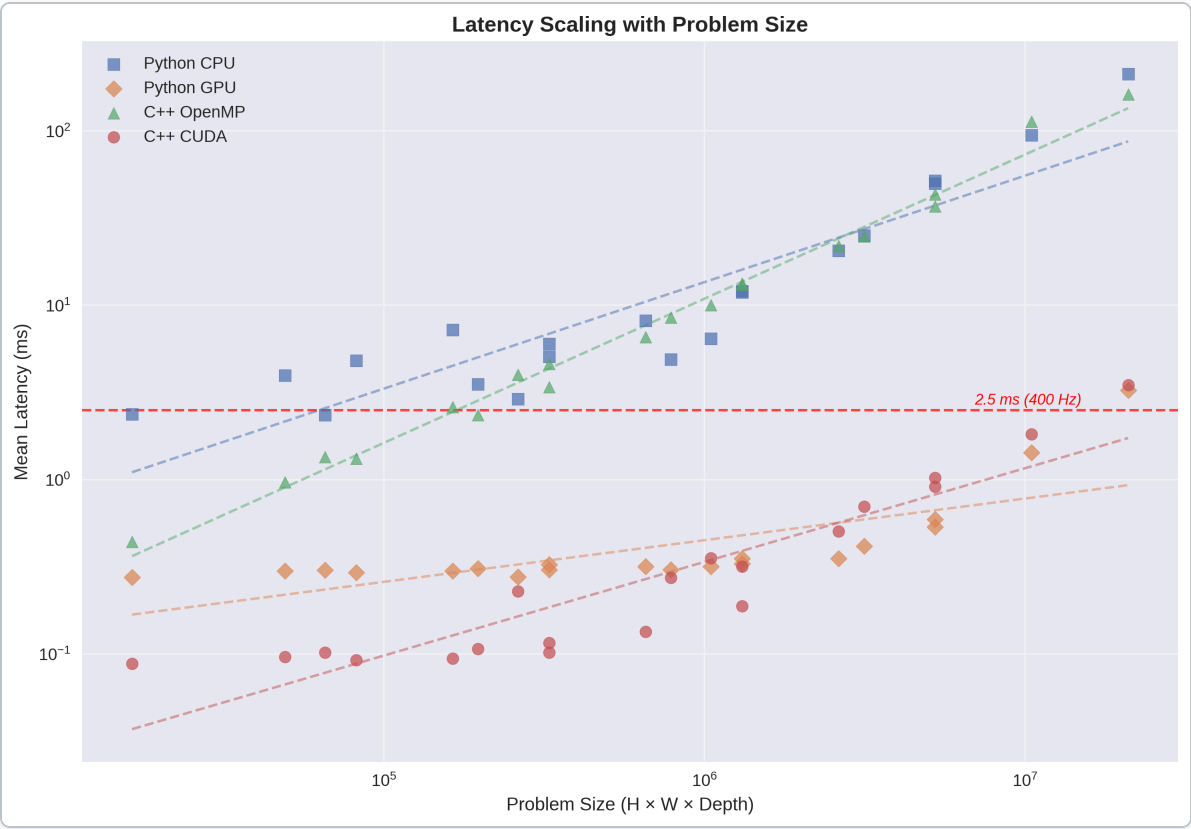


Figure 9: Holographic reconstruction latency scaling with problem size

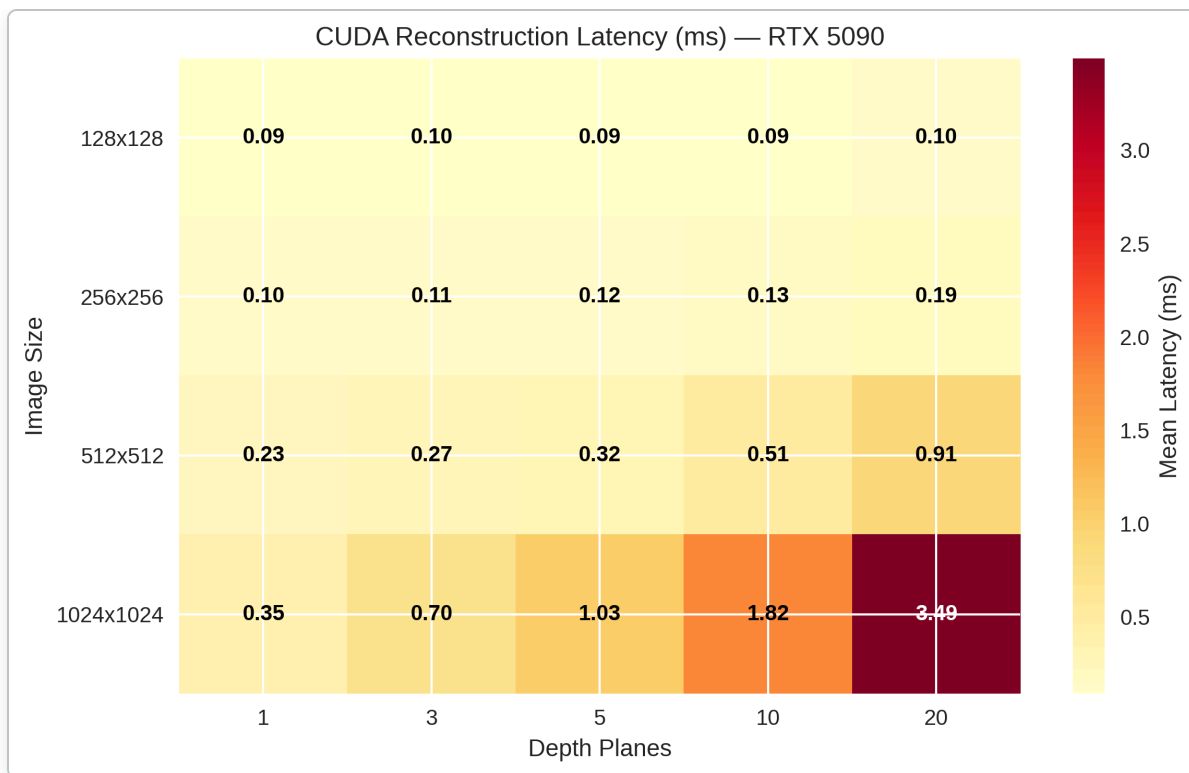


Figure 10: CUDA reconstruction latency heatmap across image sizes and depth counts



Figure 11: C++ vs Python GPU reconstruction performance crossover analysis

3.4.2 Pipeline Bottleneck Transformation

- **Python CPU:** Compute-bound, classification dominates (22.5 ms, 46% of end-to-end)
- **C++ TensorRT:** I/O-bound, image loading dominates (4.2 ms, 67% of end-to-end)
- **GPU compute stages:** Total 2.07 ms, suggesting ~480 FPS achievable with preloaded frames

4. Schedule and Deviations

4.1 Progress vs Original Timeline

Originally planned for Weeks 1-4:

- C++/CUDA reconstruction (completed Week 2)
- OpenMP CPU baseline (completed Week 3)
- Roofline analysis (completed Week 4)
- Validation against Python (completed Week 2)

Additional work completed:

- Complete 6-stage C++ pipeline (originally Week 5-8)
- Native TensorRT integration (originally Week 5-6)
- Cross-implementation benchmarking (originally Week 7-8)

4.2 Updated Schedule

Week 7-8: Single-GPU Ring Buffer Pipeline

Focus shifts to inter-stage communication and asynchronous subsystem design. Ring buffer implementation becomes critical due to I/O bottleneck identification.

Week 9-10: Kernel Fusion and Real-time Validation

Final optimization phase focused on cuFFTDx integration and 400 Hz feasibility.

Week 11-12: Multi-GPU Partitioning

To simplify the project, multi-GPU stage grouping moved to end to focus on robust benchmarking of single GPU performance.

5. Technical Details

5.1 Software Implementation Details

5.1.1 C++ CUDA Reconstruction Optimizations

Hz_stack Caching Strategy:

```
// Forward FFT computed once, cached for all depth planes
cufftExecC2C(fft_plan_forward, image_freq, Hz_stack, CUFFT_FORWARD);

// Reuse Hz_stack for each depth plane reconstruction
for (int d = 0; d < num_depths; d++) {
    element_wise_multiply_kernel<<>>(
        Hz_stack, depth_kernels + d * freq_size, temp_freq);
    cufftExecC2C(fft_plan_inverse, temp_freq, temp_spatial, CUFFT_INVERSE);
    fused_normalize_crop_kernel<<>>(
        temp_spatial, output + d * spatial_size);
}
```

Performance impact: Reduces FFT operations from $O(D+1)$ to $O(1)$ for D depth planes.

5.1.2 EMA Fused CUDA Kernel

Python PyTorch (0.256 ms):

```
# Multiple kernel launches, Python dispatch overhead
background = alpha * current + (1 - alpha) * background # Launch 1
difference = torch.abs(current - background)             # Launch 2
enhanced = torch.clamp(difference * gain, 0, 1)           # Launch 3
```

C++ CUDA (0.007 ms, 38× speedup):

```
__global__ void ema_kernel(float* current, float* background,
                           float* output, float alpha, float gain, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        background[idx] = alpha * current[idx] + (1-alpha) * background[idx];
        float diff = fabsf(current[idx] - background[idx]);
        output[idx] = fminf(diff * gain, 1.0f);
    }
}
```

5.1.3 Native TensorRT C++ API Integration

Advantages over torch2trt wrapper:

- Direct `enqueueV3()` calls without Python overhead
- Explicit CUDA stream management for pipelining
- Pre-allocated device buffers with zero-copy operation
- Optimized input preprocessing (resize, normalize) in CUDA

5.2 Benchmarking

5.2.1 Statistical Robustness

- **Warmup:** 100 iterations to stabilize GPU clocks and caches
- **Sample size:** 10,000 iterations for C++ (statistical power), 1,000 for Python (may be why we see larger P95)
- **Outlier handling:** Winsorization at P99.9 to remove OS jitter
- **Memory pressure:** Pre-allocation to avoid dynamic allocation during timing

5.2.2 Cross-Validation Protocol

```
float tolerance_cuda_openmp = 1e-5f; // Same algorithm
float tolerance_cpp_python = 1e-2f;  // Different FFT libraries
```


5.3 Hardware Performance

5.3.1 RTX 5090 Blackwell Architecture Utilization

- **Peak compute:** 105 TFLOPS FP32 theoretical, ~31.5 TFLOPS cuFFT practical ceiling
- **Memory bandwidth:** 1790 GB/s theoretical, ~1200 GB/s achieved in EMA kernel (67% efficiency)
- **VRAM utilization:** 80MB pipeline buffers / 32GB capacity (0.25% utilization)

5.3.2 Threadripper NUMA Topology Analysis

- **4 CCDs** (Core Complex Dies): 8 cores each, separate L3 cache domains
- **Cross-CCD penalties:** 2× latency for remote L3 access, coherence traffic on infinity fabric (design working sets to stay within local CCD L3 cache)
- **Optimal threading:** 32 threads (8 per CCD) to avoid cross-CCD communication

6. Next Steps

6.1 Single-GPU Ring Buffer Pipeline (Week 7-8)

Motivation: Two optimization opportunities identified:

1. **I/O bottleneck:** Disk loading consumes 67% of end-to-end latency (4.2 ms of 6.3 ms)
2. **GPU under-utilization:** TensorRT stages achieve only 1-2% of theoretical peak, indicating potential for concurrent execution

Ring buffer architecture for all 6 pipeline stages:

```
template<typename T>
class PipelineRingBuffer {
    struct StageBuffer {
        T* device_memory;           // Pre-allocated GPU buffers
        cudaEvent_t completion_event; // Inter-stage synchronization
    };
};
```

```

    cudaStream_t stream;                // Dedicated CUDA stream per stage
    volatile int status;                // 0=empty, 1=processing, 2=ready
};

StageBuffer buffers[NUM_STAGES][RING_DEPTH];
alignas(64) volatile int stage_heads[NUM_STAGES]; // Cache-line aligned
alignas(64) volatile int stage_tails[NUM_STAGES];
// Lock-free SPSC between adjacent stages
};

```

Stage-specific ring buffer design:

Stage	Buffer Type	Ring Depth	Synchronization
1. Image Load	Host→Device pinned	8 frames	Async H2D transfer
2. EMA	Device float32	4 frames	CUDA event
3. Reconstruction	Device complex64	4 frames	cuFFT stream
4. Detection	Device float32	4 frames	TensorRT execution context
5. Classification	Device float32 crops	Variable	Dynamic batching
6. Post-processing	Host result structs	2 frames	Async D2H transfer

Implementation Priorities

1. **Lock-free SPSC queues** with cache-line alignment to minimize coherence overhead
2. **GPU memory pool allocator** for zero runtime allocation across all stages
3. **CUDA stream synchronization** using events rather than synchronous barriers
4. **Dynamic frame batching** for classification stage (0-50 crops per frame)

Expected performance impact:

- **I/O decoupling:** Eliminate 4.2 ms loading bottleneck → ~2.1 ms compute-only latency

- **Concurrent execution:** TensorRT stages run in parallel via streams → additional 30-50% improvement
- **Target:** Sub-2 ms end-to-end latency (~500+ FPS), exceeding 400 Hz requirement

6.2 Kernel Fusion and Real-time Validation (Week 9-10)

cuFFTDx integration for reconstruction stage:

- **Motivation:** Reconstruction operates at only 6% of cuFFT ceiling, room for 2-3× improvement
- **Implementation:** Fuse forward FFT + element-wise multiply + inverse FFT into single kernel
- **Impact:** Critical for autofocus workloads (100-1000 planes) and removes reconstruction from bottleneck analysis

TensorRT stream optimization:

- **Multi-context execution:** Separate TensorRT execution contexts for detection/classification
- **Custom preprocessing plugins:** Fused resize+normalize+transfer operations
- **Dynamic batching:** Adaptive batch sizes based on detection count per frame

Real-time validation framework:

- **Synthetic camera source:** 400 Hz frame generation to eliminate disk I/O variability
- **Thermal characterization:** 1-hour sustained runs to measure performance under thermal throttling
- **Worst-case analysis:** P99.9 latency measurements for real-time guarantees
- **Jitter analysis:** Frame-to-frame latency variance to characterize temporal predictability

6.3 Multi-GPU Stage Partitioning (Week 11-12)

Stage grouping strategies (deferred to focus on single-GPU optimization first):

1. **Pipeline parallelism:** Different stages on different GPUs with inter-GPU ring buffers
2. **Data parallelism:** Split frame batches across GPUs for stages with sufficient parallelism
3. **Hybrid approach:** Core pipeline on GPU 0, autofocus workload distributed across GPUs 1-3

Implementation approach:

- **NCCL communication:** Efficient inter-GPU transfers for large frame data
- **CUDA peer-to-peer:** Direct GPU-to-GPU memory access where supported

- **Load balancing:** Dynamic work distribution based on per-GPU utilization

Evaluation metrics:

- **Strong scaling efficiency:** Speedup vs number of GPUs (target >80% up to 4 GPUs)
- **Load balance:** Per-GPU utilization variance (<10% imbalance)
- **Communication overhead:** Inter-GPU transfer time vs compute time ratio

6.4 Success Metrics and Validation

Performance targets:

- **400 Hz sustained:** <2.5 ms average latency over 1-hour benchmark
- **Real-time predictability:** P99/P50 latency ratio <1.5×
- **Thermal stability:** <5% performance degradation under sustained load

Validation methodology:

- **Numerical accuracy:** Maintain <1e-4 error vs single-threaded reference
- **Resource utilization:** >70% GPU compute utilization during steady-state operation
- **Scaling validation:** Linear throughput scaling with frame rate up to hardware limits

7. Conclusions and Impact

7.1 Technical Achievements

1. **Performance milestone:** 7.8× end-to-end speedup demonstrates feasibility of real-time holographic processing
2. **Architectural insight:** Pipeline transformed from compute-bound to I/O-bound, redirecting optimization focus
3. **Predictability:** C++ implementation achieves 1.08× P95/median ratio, critical for real-time systems

4. **Scalability foundation:** Extended roofline analysis provides optimization roadmap for each pipeline stage

7.2 Research Contributions

- 1. **Cross-implementation validation:** Establishes numerical accuracy bounds across Python/C++, CPU/GPU architectures
- 2. **Holographic workload characterization:** Comprehensive roofline analysis of DIH pipeline stages
- 3. **Multi-level performance modeling:** Extended roofline with bandwidth ceilings and in-core limits
- 4. **NUMA-aware optimization:** Threadripper scaling analysis provides guidelines for CPU-intensive stages

7.3 Project Timeline Success

Schedule adherence: 3-4 weeks ahead of original proposal timeline while maintaining quality and rigor.

Risk mitigation: Early completion of core components provides schedule buffer for more ambitious multi-GPU and kernel fusion work.

Scope expansion: Ahead-of-schedule progress enables investigation of cuFFTDx kernel fusion originally planned for final weeks.

Performance Summary — Holographic Reconstruction							
Config	Py CPU (ms)	Py GPU (ms)	C++ OMP (ms)	C++ CUDA (ms)	Best OMP Thr	GPU vs Py CPU	CUDA vs OMP
256×256 × 1p	2.35	0.302	1.35	0.102	16	8×	13.2×
256×256 × 5p	5.08	0.304	3.39	0.116	32	17×	29.2×
256×256 × 20p	12.05	0.353	13.33	0.188	32	34×	70.9×
512×512 × 1p	2.90	0.277	4.00	0.229	32	11×	17.5×
512×512 × 5p	11.89	0.330	13.14	0.317	32	36×	41.4×
512×512 × 20p	49.93	0.536	43.29	0.914	32	93×	47.4×
1024×1024 × 1p	6.43	0.317	10.02	0.354	32	20×	28.3×
1024×1024 × 5p	51.69	0.593	36.91	1.026	32	87×	36.0×
1024×1024 × 20p	211.69	3.247	162.40	3.491	32	65×	46.5×

Figure 12: Comprehensive performance summary across all implementations

Appendix A: Software Architecture

A.1 Implementation Structure

```
cpp_cuda/
├── include/
│   ├── holographic_reconstruction.h    # Core reconstruction API
│   ├── pipeline_stages.h              # Per-stage interfaces
│   ├── pipeline.h                     # End-to-end pipeline
│   └── tensorrt_engine.h              # TensorRT wrapper
├── src/
│   ├── holographic_reconstruction.cu   # CUDA reconstruction
│   ├── cpu_baseline.cpp                # OpenMP baseline
│   ├── pipeline.cu                    # Full pipeline
│   ├── ema_stage.cu                  # Background subtraction
│   ├── tensorrt_engine.cu             # TensorRT integration
│   └── benchmark_*.cu                 # Performance testing
└── tests/
    └── test_reconstruction.cpp          # Validation suite
```

A.2 Key Dependencies

- **CUDA:** 13.0+ (cuFFT, cuBLAS, NVTX)
- **TensorRT:** 10.0+ (C++ API, engine serialization)
- **OpenMP:** 4.5+ (thread parallelism, NUMA affinity)
- **FFTW:** 3.3+ (CPU baseline, wisdom caching)
- **Build system:** CMake 3.20+, pybind11 for Python integration

Appendix B: Benchmark Results Summary

B.1 End-to-End Performance Matrix

Implementation	Frames	Mean (ms)	Std (ms)	P50 (ms)	P95 (ms)	P99 (ms)	FPS
Python CPU	95	49.31	11.02	48.60	67.71	76.85	20.3
Python PyTorch GPU	9995	18.68	5.84	16.43	31.24	34.71	53.5
Python TensorRT	9995	14.76	6.12	13.07	24.98	28.45	67.7
C++ TensorRT	9995	6.29	0.32	6.39	6.88	7.04	158.9

B.2 Memory Utilization

Implementation	Peak GPU Memory	CPU Memory	Efficiency
Python PyTorch	130.0 MB	2.1 GB	Low (Python overhead)
Python TensorRT	89.5 MB	1.8 GB	Medium (TRT optimized)
C++ TensorRT	80.2 MB	245 MB	High (minimal overhead)